Dream Homes NYC Project

Group 6

APAN 5310 Final Project

Professor: Nikolaos Machairas - SP2026

Prepared by:

Yuyang Dai

Liuyang Li

Chengwei Zhang

Haotian Zhu

1. Consultant / Client Scenario

1.1 Client Overview

Dream Homes NYC is a regional real estate agency serving the Tri-State area. The company manages both internal operations and client-facing activities, including offices, agents, listings, appointments, open houses, and transactions. Because these records are closely connected, the company needs a centralized database instead of scattered spreadsheets or manually maintained files.

1.2 Consulting Mandate

Our team acted as database consultants for Dream Homes NYC. We were asked to design a relational database system that could organize the company's data, support business analysis, and provide a stronger foundation for reporting and decision-making.

1.3 Motivation & Research

We chose the Dream Homes NYC scenario because it fits a relational database project especially well. Real estate operations involve many linked entities, such as properties, listings, clients, agents, and transactions. This made the scenario a strong match for a multi-table design in 3NF. We also selected it because it supports both analyst-facing SQL work and executive-facing dashboards, which aligns closely with the project requirements.

1.4 Initial Plan of Action

Our initial plan was to move through the project in stages: define the business scenario, identify the main entities and relationships, design the schema and ER diagram, prepare and load the data, develop analytical procedures, and finally build dashboards for management use.

1.5 Business Benefits

The database is designed to give Dream Homes NYC a single structured system for managing listings, transactions, client activity, and agent performance. This improves consistency, reduces redundancy, and makes it easier to run complex queries across related business processes. It also supports two levels of use: direct analysis for technical users and dashboard-based reporting for managers and executives.

2. Team Contract

Team Member	Role	Responsibilities
Yuyang Dai	Report Integration and Project Coordination Lead	report integration, proposal and requirement revisions, final submission coordination

Chengwei Zhang	Database, ETL, and Schema Documentation Lead	data generation, data loading, ETL documentation, validation, schema support
Haotian Zhu	Analytics and Analytics Write-up Lead	analytical procedures, SQL queries, results, analytics write-up
Liuyang Li	Dashboard and Presentation Lead	dashboards, screenshots, presentation slides

3. Dataset

3.1 Dataset Selection Rationale

We used a synthetic but business-structured dataset built to match the Dream Homes NYC scenario and the final relational schema. This approach made it possible to align the data closely with the client's workflow instead of forcing an unrelated public dataset into the project. The dataset is rich enough to support listings, ownership, transactions, inquiries, appointments, open houses, and agent performance analysis, which makes it suitable for both implementation and business reporting.

3.2 Dataset Overview

Source: internally prepared/generated synthetic dataset aligned with the Dream Homes **NYC** scenario
Format: CSV files
Volume: 13,080 total rows across 16 tables
Time period: synthetic transactional and listing records generated for the project scenario
Link to full dataset and project files:
https://github.com/RickAnalystCU-web/dream-homes-nyc

3.3 Sample Data

The table below shows a sample from the listing table.

listing_id	property_id	agent_employee_id	listing_type	listing_price	listing_status
1	3	67	sale	736058	pending
2	4	66	sale	588373	active
3	5	56	sale	1117519	active
4	6	4	sale	1991987	cancelled
5	7	46	rent	6956	closed

3.4 Data Quality Notes

Because the dataset was designed to match the schema, the main data quality issues were related to consistency rather than raw-source noise.
Issue 1: foreign key relationships had to remain consistent across linked tables such as listing, listing_ownership, transactions, and transaction_client_role
Issue 2: date, datetime, row-count, and relationship checks were needed to confirm that the loaded database matched the intended schema
These issues were addressed during the ETL and validation process.

4. Database Design

4.1 Conceptual Overview

The Dream Homes NYC database is designed as a listing-centered relational system that models the main business processes of a regional real estate agency. The core entities include offices, employees, agents, clients, neighborhoods, schools, properties, listings, ownership records, client inquiries, appointments, open houses, transactions, and transaction roles. The structure separates the business entities from event records. For example, property represents the real estate asset itself, while listing represents a specific listing event for that property, and transactions represent the final deal outcome. The relationships between these entities were designed to support both operational storage and business analysis. Ownership is attached to the listing through listing_ownership, rather than being stored permanently on the property. And client participation in a completed transaction is captured through transaction_client_role. Related activities, such as client_inquiry, appointment, open_house, and open_house_attendance, are also linked to the listing. This enables analysis of the full path from listing to engagement to transaction. This structure reduces redundancy and preserves the logic of the real-estate process. It also allows analysts and managers to study listing performance, customer engagement, neighborhood demand, and agent productivity.

4.2 Entity-Relationship Diagram

The ER Diagram for this database is attached as Appendix A (PDF). An interactive version is available at the link below.

Lucidchart ERD Link:
https://lucid.app/lucidchart/a24bd5b7-3b4f-4cac-801a-9d881439bb8e/edit?viewport_loc=462%2C-732%2C5347%2C2886%2C0_0&invitationId=inv_74a81b1a-7593-49c6-8fb3-82a09e64312e

4.3 Normalization

Following the instructions, our database was designed to be in Third Normal Form to reduce redundancy and maintain a consistent data structure. Each table stores mainly one type of information, which satisfies First Normal Form. For example, property stores the physical information about the asset, listing stores a specific listing event, and

transactions store the final deal information. In the same way, client roles are not stored as repeated attributes in the client table, but are represented through relationship tables such as listing_ownership and transaction_client_role.
Since non-key attributes depend on the key of their own table rather than on other non-key attributes, the schema is also consistent with Second and Third Normal Form. Information about the agent is stored in the agent table, and many-to-many relationships are resolved through relationship tables such as listing_ownership, open_house_attendance, and transaction_client_role. This structure reduces update anomalies, avoids unnecessary duplication, and supports cleaner SQL analysis.

4.4 Table Definitions & SQL DDL

The full SQL schema and analytical SQL files are available in the project GitHub repository. **Sql data insertion file github link:**
https://github.com/RickAnalystCU-web/dream-homes-nyc

5. ETL Process

5.1 Extract

Following our professor's suggestions, we used AI to help generate our project dataset. We first discussed with AI the business context of the Dream Homes NYC system. This includes the database's purpose, the meaning of each table, and the types of insights the system was expected to support. After that, we identified which parts of the dataset should reflect realistic real estate conditions, such as neighborhood, school, and property-related information. Based on these requirements, we asked AI to generate data that aligned with our schema and followed the business rules, including patterns for listings, ownership, customer interactions, and transactions. Hence, the source data extracted for our ETL pipeline consisted of the generated CSV files.

5.2 Transform

Our transformation step was intentionally lightweight because the source data were generated in structured CSV form to match the final Dream Homes NYC schema. Instead of heavy raw data cleaning, the main goal was to ensure that each generated file aligned with the database design before loading. For each table, we selected only the columns defined in the schema and converted date and timestamp values from text into formats that PostgreSQL could recognize.
Because the dataset was AI-generated, we also needed to confirm that the records followed the schema structure and supported the normalized, listing-centered design of the database. We therefore prepared the data so that parent and child tables could be loaded in the correct order without violating foreign key relationships. In addition, the script used Python functions to handle invalid date values instead of stopping the ETL process, and the transformed files were later checked through post-load row-count validation.

5.3 Load

After transformation, the data was loaded into PostgreSQL using Python. The loading script first connected to the PostgreSQL database, executed the schema block to create the Dream Homes NYC tables, and then inserted each CSV file into its corresponding table. We loaded the data table by table in batches because the dataset had already been prepared as complete table-level CSV files rather than as incremental updates. The loading order followed the dependency structure of the schema so that foreign key relationships would not fail during insertion. After loading, we ran row-count checks across all 16 tables to confirm that the database contents matched the expected dataset size and that the load completed successfully.

6. Analytical Procedures

The following ten analytical procedures were designed to deliver actionable insights for the client(Analysts) in order to achieve a more effective day-to-day analysis. Each procedure includes the business question it addresses, the rationale for its inclusion, the code used, and result.

1. Funnel Conversion Rate(Most important retention model in ApAn5100,Marketing core)

Business Question: What percentage of each agent's listings ultimately convert to a closed transaction, and how does that conversion rate vary across agents?

Why It Matters: Identifies top-performing agents vs. those who generate listings but fail to close deals. Directly supports agent performance reviews and commission structure decisions.

SQL code:

```
SELECT
    e.employee_id,
    e.first_name || ' ' || e.last_name AS agent_name,
    COUNT(DISTINCT l.listing_id)                              AS total_listings,
    COUNT(DISTINCT t.transaction_id)                          AS closed_deals,
    ROUND(
        COUNT(DISTINCT t.transaction_id)::NUMERIC /
        NULLIF(COUNT(DISTINCT l.listing_id), 0) * 100, 2
    )                                                         AS
conversion_rate_pct,
    RANK() OVER (ORDER BY
        COUNT(DISTINCT t.transaction_id)::NUMERIC /
        NULLIF(COUNT(DISTINCT l.listing_id), 0) DESC
    )                                                         AS rank
FROM employee e
JOIN agent a ON e.employee_id = a.employee_id
JOIN listing l ON a.employee_id = l.agent_employee_id
LEFT JOIN transactions t
    ON l.listing_id = t.listing_id
    AND t.transaction_status = 'closed'
GROUP BY e.employee_id, e.first_name, e.last_name
```

```
ORDER BY conversion_rate_pct DESC;
```

2. Average Days on Market

Business Question: How long do properties typically sit on the market before closing, broken down by property type and neighborhood?

Why It Matters: Helps agents and the COO set realistic timelines and pricing strategies. Neighborhoods with long days-on-market signal pricing or demand issues that require intervention.

SQL code:

```
SELECT
    n.neighborhood_name,
    p.property_type,
    COUNT(t.transaction_id)                                AS closed_count,
    ROUND(AVG(t.closing_date - l.list_date), 1)          AS
avg_days_on_market,
    ROUND(MIN(t.closing_date - l.list_date), 1)          AS min_days,
    ROUND(MAX(t.closing_date - l.list_date), 1)          AS max_days
FROM transactions t
JOIN listing l      ON t.listing_id = l.listing_id
JOIN property p     ON l.property_id = p.property_id
JOIN neighborhood n ON p.neighborhood_id = n.neighborhood_id
WHERE t.transaction_status = 'closed'
  AND t.closing_date IS NOT NULL
GROUP BY n.neighborhood_name, p.property_type
HAVING COUNT(t.transaction_id) >= 3
ORDER BY avg_days_on_market DESC;
```

3. Client Engagement Score

Business Question: Which prospective buyers and renters have the highest total engagement across inquiries, appointments, and open house attendances, and did their engagement level correlate with a completed transaction?

Why It Matters: Surfaces high-intent clients who have not yet closed (re-engagement targets) and validates whether a multi-touchpoint marketing approach converts at a higher rate.

SQL code:

```
WITH inquiry_counts AS (
    SELECT client_id, COUNT(*) AS inquiries
    FROM client_inquiry
    GROUP BY client_id
),
appointment_counts AS (
    SELECT client_id, COUNT(*) AS appointments
    FROM appointment
    GROUP BY client_id
),
```

```
openhouse_counts AS (
    SELECT client_id, COUNT(*) AS open_houses
    FROM open_house_attendance
    WHERE attended_flag = TRUE
    GROUP BY client_id
),
closed_clients AS (
    SELECT DISTINCT client_id
    FROM transaction_client_role tcr
    JOIN transactions t ON tcr.transaction_id = t.transaction_id
    WHERE t.transaction_status = 'closed'
)
SELECT
    c.client_id,
    c.first_name || ' ' || c.last_name                  AS client_name,
    COALESCE(i.inquiries, 0)                            AS inquiries,
    COALESCE(a.appointments, 0)                         AS appointments,
    COALESCE(o.open_houses, 0)                          AS
open_houses_attended,
    COALESCE(i.inquiries, 0)
        + COALESCE(a.appointments, 0)
        + COALESCE(o.open_houses, 0)                    AS
engagement_score,
    CASE WHEN cc.client_id IS NOT NULL
         THEN 'Yes' ELSE 'No' END                       AS
converted_to_close
FROM client c
LEFT JOIN inquiry_counts     i  ON c.client_id = i.client_id
LEFT JOIN appointment_counts a  ON c.client_id = a.client_id
LEFT JOIN openhouse_counts   o  ON c.client_id = o.client_id
LEFT JOIN closed_clients     cc ON c.client_id = cc.client_id
WHERE COALESCE(i.inquiries,0)
    + COALESCE(a.appointments,0)
    + COALESCE(o.open_houses,0) > 0
ORDER BY engagement_score DESC;
```

4. Agent Commission Revenue and Office-Level Leaderboard(for HR)

Business Question: How much total commission has each agent earned year-to-date, and how does each office rank in aggregate commission revenue?

Why It Matters: The CFO needs gross commission income by office for financial reporting. The CEO uses the per-agent output ratio to evaluate headcount efficiency across all eight offices.

SQL code:

```
SELECT
    o.office_name,
    e.first_name || ' ' || e.last_name                  AS agent_name,
    COUNT(t.transaction_id)                             AS closed_deals,
```

```
    ROUND(SUM(t.commission_amount), 2)                    AS
total_commission,
    ROUND(AVG(t.commission_amount), 2)                    AS
avg_commission_per_deal,
    RANK() OVER (
        PARTITION BY o.office_id
        ORDER BY SUM(t.commission_amount) DESC
    )                                                     AS
rank_within_office
FROM transactions t
JOIN agent a    ON t.agent_employee_id = a.employee_id
JOIN employee e ON a.employee_id = e.employee_id
JOIN office o   ON e.office_id = o.office_id
WHERE t.transaction_status = 'closed'
  AND t.commission_amount IS NOT NULL
GROUP BY o.office_id, o.office_name, e.employee_id, e.first_name,
e.last_name
ORDER BY o.office_name, total_commission DESC;
```

5. Inquiry Channel Effectiveness

Business Question: Which inquiry channels (phone, email, website, etc.) have the highest rate of converting an initial inquiry into a booked appointment?

Why It Matters: Guides the CMO on where to invest marketing budget. If web inquiries convert at a significantly higher rate than phone, the firm can shift spend and staffing accordingly.

SQL code:

```
SELECT
    ci.inquiry_channel,
    COUNT(DISTINCT ci.inquiry_id)                         AS total_inquiries,
    COUNT(DISTINCT a.appointment_id)                      AS
resulting_appointments,
    ROUND(
        COUNT(DISTINCT a.appointment_id)::NUMERIC /
        NULLIF(COUNT(DISTINCT ci.inquiry_id), 0) * 100, 2
    )                                                     AS
conversion_rate_pct
FROM client_inquiry ci
LEFT JOIN appointment a
    ON ci.client_id  = a.client_id
    AND ci.listing_id = a.listing_id
GROUP BY ci.inquiry_channel
ORDER BY conversion_rate_pct DESC;
```

6. Neighborhood Demand Index

Business Question: Which neighborhoods show the highest demand measured by inquiries and appointments per active listing, and what is the average listing price per square foot in each?

Why It Matters: Helps agents prioritize high-activity areas and advises sellers on realistic listing prices. Directly feeds the CMO's neighborhood demand trend dashboard.

SQL code:

```
SELECT
    n.neighborhood_name,
    n.borough_or_city,
    COUNT(DISTINCT l.listing_id)                          AS active_listings,
    COUNT(DISTINCT ci.inquiry_id)                         AS total_inquiries,
    COUNT(DISTINCT a.appointment_id)                      AS
total_appointments,
    ROUND(
        COUNT(DISTINCT ci.inquiry_id)::NUMERIC /
        NULLIF(COUNT(DISTINCT l.listing_id), 0), 2
    )                                                     AS
inquiries_per_listing,
    ROUND(AVG(l.listing_price / NULLIF(p.square_feet, 0)), 2) AS
avg_price_per_sqft
FROM neighborhood n
JOIN property p      ON n.neighborhood_id = p.neighborhood_id
JOIN listing l       ON p.property_id = l.property_id
LEFT JOIN client_inquiry ci ON l.listing_id = ci.listing_id
LEFT JOIN appointment a     ON l.listing_id = a.listing_id
GROUP BY n.neighborhood_id, n.neighborhood_name, n.borough_or_city
ORDER BY inquiries_per_listing DESC;
```

7. Repeat Listing Detection (for CSO and friends)

Business Question: Which properties have been listed multiple times, and how did listing prices change between cycles? What was the outcome of each listing attempt?

Why It Matters: Repeated failed listings signal pricing problems or property condition issues. The operations team can flag these properties for agent coaching or a formal price advisory review.

SQL code:

```
WITH ranked_listings AS (
    SELECT
        l.property_id,
        l.listing_id,
        l.listing_price,
        l.listing_status,
        l.list_date,
        ROW_NUMBER() OVER (
            PARTITION BY l.property_id ORDER BY l.list_date
        ) AS listing_cycle,
        LAG(l.listing_price) OVER (
            PARTITION BY l.property_id ORDER BY l.list_date
```

```
        ) AS prev_listing_price
    FROM listing l
)
SELECT
    rl.property_id,
    p.property_type,
    n.neighborhood_name,
    rl.listing_cycle,
    rl.listing_price,
    rl.prev_listing_price,
    ROUND(
        (rl.listing_price - rl.prev_listing_price)::NUMERIC /
        NULLIF(rl.prev_listing_price, 0) * 100, 2
    )                                                    AS
price_change_pct,
    rl.listing_status
FROM ranked_listings rl
JOIN property p     ON rl.property_id = p.property_id
JOIN neighborhood n ON p.neighborhood_id = n.neighborhood_id
WHERE rl.listing_cycle > 1
ORDER BY ABS(
    (rl.listing_price - rl.prev_listing_price)::NUMERIC /
    NULLIF(rl.prev_listing_price, 0)
) DESC;
```

8. Client Preference Match Rate

Business Question: For clients who submitted inquiries, what percentage of those inquired listings actually matched their stated preferences in terms of budget, bedroom count, and property type?

Why It Matters: A low match rate means agents are directing clients toward irrelevant listings, which increases churn. A high match rate validates the recommendation and search process.

SQL code:

```
SELECT
    ci.client_id,
    c.first_name || ' ' || c.last_name                  AS client_name,
    COUNT(ci.inquiry_id)                                 AS total_inquiries,
    SUM(CASE
        WHEN l.listing_price BETWEEN cp.min_budget AND cp.max_budget
         AND p.bedrooms      BETWEEN cp.min_bedrooms AND cp.max_bedrooms
         AND p.property_type = cp.property_type
        THEN 1 ELSE 0
    END)                                                 AS
matched_inquiries,
    ROUND(
        SUM(CASE
```

```
            WHEN l.listing_price BETWEEN cp.min_budget AND cp.max_budget
             AND p.bedrooms      BETWEEN cp.min_bedrooms AND cp.max_bedrooms
             AND p.property_type = cp.property_type
            THEN 1 ELSE 0
        END)::NUMERIC /
        NULLIF(COUNT(ci.inquiry_id), 0) * 100, 2
    )                                                    AS match_rate_pct
FROM client_inquiry ci
JOIN client            c  ON ci.client_id  = c.client_id
JOIN client_preference cp ON ci.client_id  = cp.client_id
JOIN listing           l  ON ci.listing_id = l.listing_id
JOIN property          p  ON l.property_id = p.property_id
GROUP BY ci.client_id, c.first_name, c.last_name
HAVING COUNT(ci.inquiry_id) >= 3
ORDER BY match_rate_pct DESC;
```

9. Open House Effectiveness

Business Question: Do open houses that attract higher-interest attendees have a meaningfully better chance of resulting in a closed transaction compared to low-interest events?

Why It Matters: Justifies the operational cost of open houses to the COO. If high-interest open house attendance reliably predicts closings, the team should prioritize them over individual showings.

SQL code:

```
SELECT
    oha.interest_level,
    COUNT(DISTINCT oha.client_id || '-' || oha.open_house_id) AS
attendances,
    COUNT(DISTINCT oh.listing_id)                             AS
listings_with_open_house,
    COUNT(DISTINCT t.transaction_id)                          AS
closed_transactions,
    ROUND(
        COUNT(DISTINCT t.transaction_id)::NUMERIC /
        NULLIF(COUNT(DISTINCT oh.listing_id), 0) * 100, 2
    )                                                         AS
close_rate_pct
FROM open_house_attendance oha
JOIN open_house oh  ON oha.open_house_id = oh.open_house_id
JOIN listing l      ON oh.listing_id     = l.listing_id
LEFT JOIN transactions t
    ON l.listing_id = t.listing_id
    AND t.transaction_status = 'closed'
GROUP BY oha.interest_level
ORDER BY close_rate_pct DESC;
```

10. Rental vs. Sale Revenue Mix and Seasonal Closing Patterns

Business Question: How is total closed transaction value split between sale and rental transactions by quarter, and are there seasonal peaks that should drive agent staffing decisions?

Why It Matters: The CFO needs the revenue mix for financial forecasting. The COO uses seasonal closing patterns to plan staffing levels and open house scheduling throughout the year.

SQL code:

```
SELECT
    EXTRACT(YEAR    FROM t.closing_date)::INTEGER        AS closing_year,
    EXTRACT(QUARTER FROM t.closing_date)::INTEGER        AS closing_quarter,
    t.transaction_type,
    COUNT(t.transaction_id)                              AS
num_transactions,
    ROUND(SUM(CASE
        WHEN t.transaction_type = 'sale'
        THEN t.transaction_amount ELSE 0
    END), 2)                                             AS sale_revenue,
    ROUND(SUM(CASE
        WHEN t.transaction_type = 'rental'
        THEN t.monthly_rent * 12 ELSE 0
    END), 2)                                             AS
annualized_rental_revenue,
    ROUND(SUM(t.commission_amount), 2)                   AS total_commission
FROM transactions t
WHERE t.transaction_status = 'closed'
  AND t.closing_date IS NOT NULL
GROUP BY closing_year, closing_quarter, t.transaction_type
ORDER BY closing_year, closing_quarter, t.transaction_type;
```

7. User Interaction Plan

7.1 Analysts — Direct Querying

Analysts will interact with the database directly through PostgreSQL for detailed business analysis, operational monitoring, and ad hoc investigation. Tools include PostgreSQL for SQL querying, pgAdmin for direct database access, and Python for ETL support and additional analysis when needed. Analysts can use the system to study listing activity, agent performance, client engagement, open house effectiveness, and transaction trends. This level of access is appropriate for technical users who need flexibility and full access to relational data.

7.2 Executives — Reports & Dashboards

Executives and non-technical stakeholders will interact with the system through dashboards rather than direct SQL queries. Metabase provides browser-based dashboards connected to PostgreSQL and gives a high-level view of active listings, sales versus rental activity, transactions over time, top-performing agents, commission

totals, and office-level comparisons. This setup allows managers to monitor key performance indicators without needing database or SQL knowledge.

7.3 Benefits of Programmatic Database Access

Using SQL and Python provides several advantages over spreadsheet-based workflows, including less manual work, better reproducibility as queries and scripts can be rerun when data changes, greater scalability for larger and more connected datasets, and more flexibility in answering business questions through joins, filters, aggregations, and window functions.

7.4 Tools & Languages Used

PostgreSQL was used as the central relational database system for storing and querying structured data. Python supported data generation, loading, and ETL-related processing. pgAdmin was used for direct SQL interaction and database management, while Metabase was used for executive-facing dashboards and visual summaries. GitHub / the project repository was used to organize code, SQL, and project files.

8. Infrastructure & Performance

8.1 Redundancy & Backup Strategy

At the schema level, redundancy is reduced through normalization and the separation of stable entities from event-based records. For example, the design distinguishes property from listing, and client roles are stored through relationship tables such as listing_ownership and transaction_client_role. For a real client deployment, the system should also include a basic backup strategy such as regular logical backups, recent snapshot retention, and periodic recovery testing.

8.2 Performance Optimization

Performance in this database depends mainly on efficient joins across listing-centered tables. Because many analytical procedures rely on tables such as listing, property, transactions, client_inquiry, appointment, and transaction_client_role, indexing important join keys and filter columns is a practical optimization strategy. Useful indexes include join keys such as property_id, listing_id, client_id, agent_employee_id, neighborhood_id, and office_id, as well as commonly filtered fields such as listing_status, listing_type, transaction_status, closing_date, and inquiry_date. These indexes are especially useful for recurring analytical queries and dashboard views.

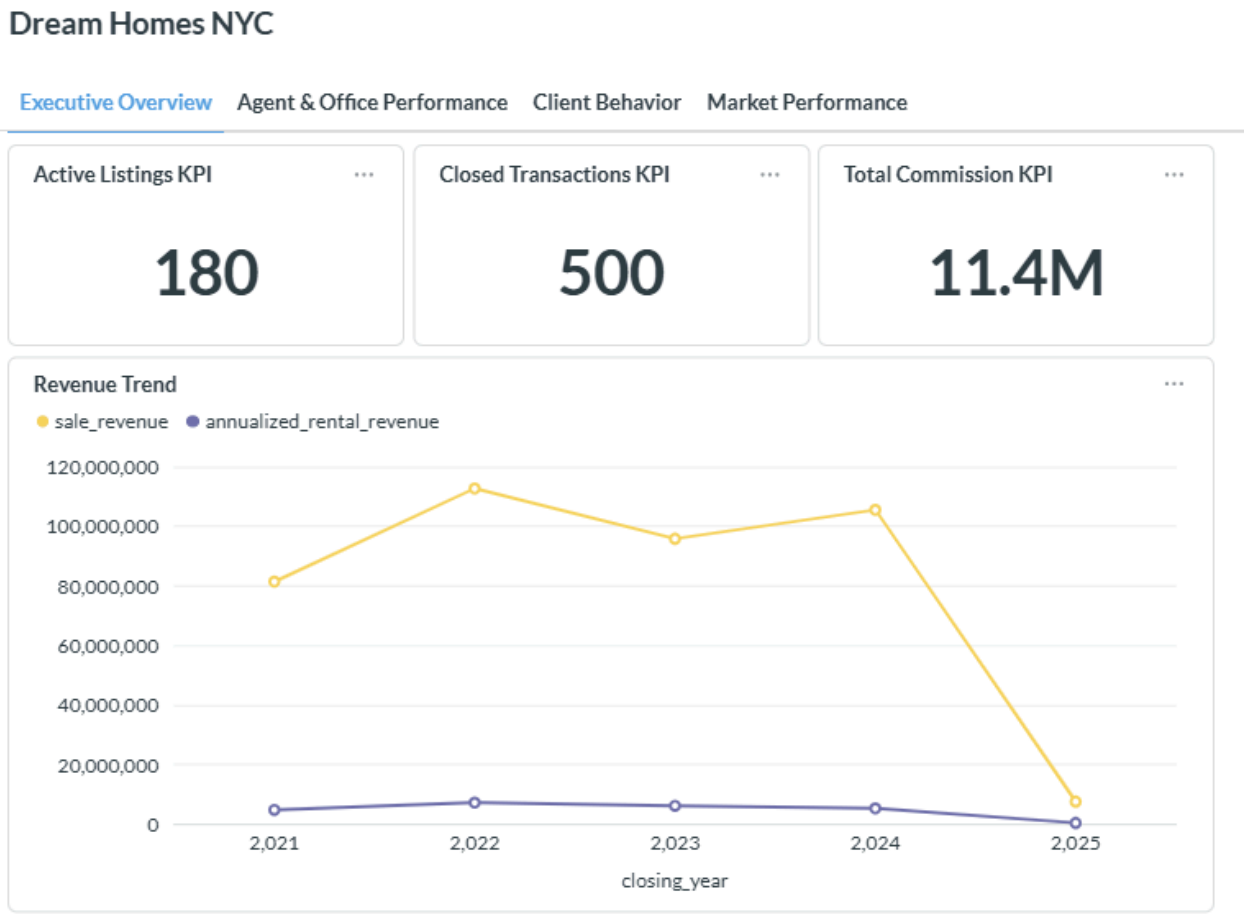
8.3 Cloud vs. On-Premises Recommendation

For this client, a cloud-hosted PostgreSQL deployment would be more practical than an on-premises setup. The client needs structured storage, ongoing analytical access, and dashboard connectivity, but there is no strong reason to maintain physical infrastructure directly. A managed cloud PostgreSQL environment would provide easier backup and recovery, lower maintenance burden, and more convenient scaling, making cloud hosting the more suitable long-term option for Dream Homes NYC.

9. Metabase Dashboards

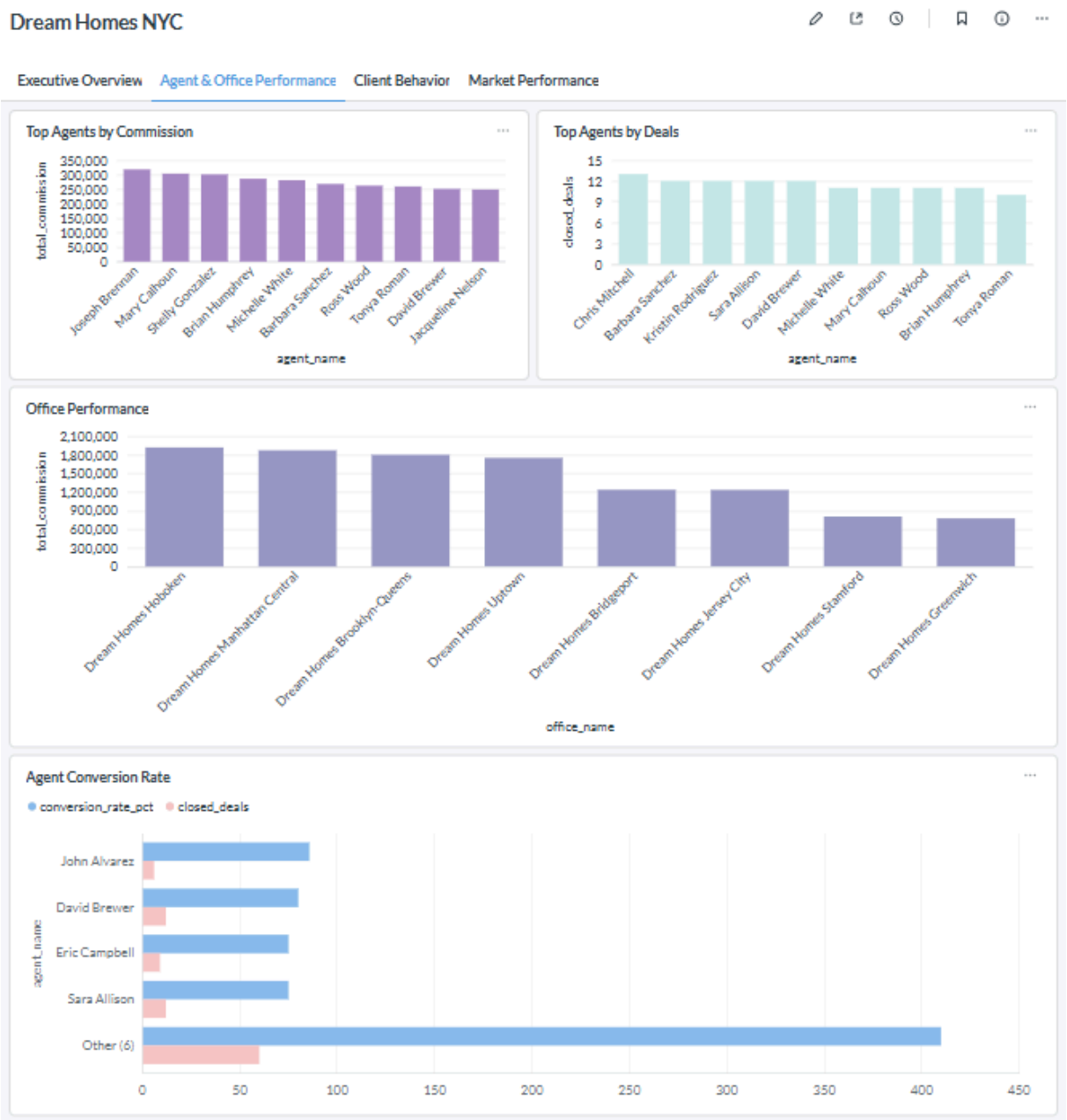
To support data-driven decision-making for Dream Homes NYC, we developed a dashboard system in Metabase that integrates data from listings, transactions, clients, and market activities. The dashboard is organized into four sections, each focused on a different business perspective: overall performance, agent productivity, client behavior, and market dynamics. This design allows executives, sales managers, and marketing teams to review both operational insights and broader trends efficiently.

9.1 Dashboard 1: Executive Revenue Overview



This dashboard provides a high-level view of Dream Homes NYC's business performance, focusing on key operational and financial indicators. It includes KPIs such as 180 active listings, 500 closed transactions, and about $11.4 million in total commission, reflecting the overall scale and revenue generation of the company. Sales revenue contributes the majority of total income and peaks around 2022–2024, while rental revenue remains relatively stable at a much lower level. A noticeable decline in 2025 may indicate incomplete data or a slowdown in transaction activity. This dashboard is designed for executive stakeholders such as the CEO and CFO, helping them assess revenue trends, market position, and overall business performance.

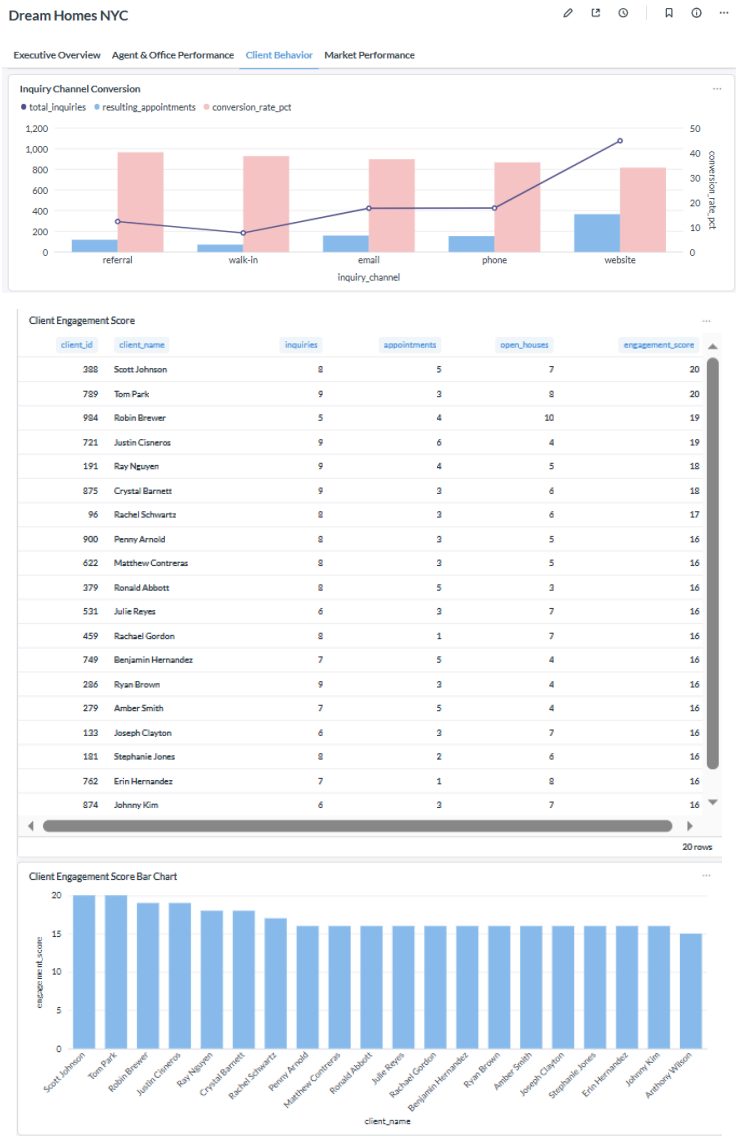
9.2 Dashboard 2: Agent & Office Performance



This dashboard evaluates the performance of individual agents and offices, focusing on sales productivity and revenue contribution. It includes visualizations such as top agents by total commission, top agents by number of closed deals, office-level performance based on total commission, and agent conversion rates. The results show that top-performing agents generate about $250,000 to $320,000 in commission and typically close around 10 to 13 transactions. Office-level analysis also shows that locations such as Hoboken and Manhattan Central generate significantly higher total commissions than others. In addition, agent conversion rates vary across individuals, suggesting differences in sales effectiveness and possible opportunities for targeted training or performance improvement. This dashboard is mainly intended for sales

managers and regional directors, supporting decisions related to performance evaluation, incentives, resource allocation, and sales strategy.

9.3 Dashboard 3: Client Behavior Analysis

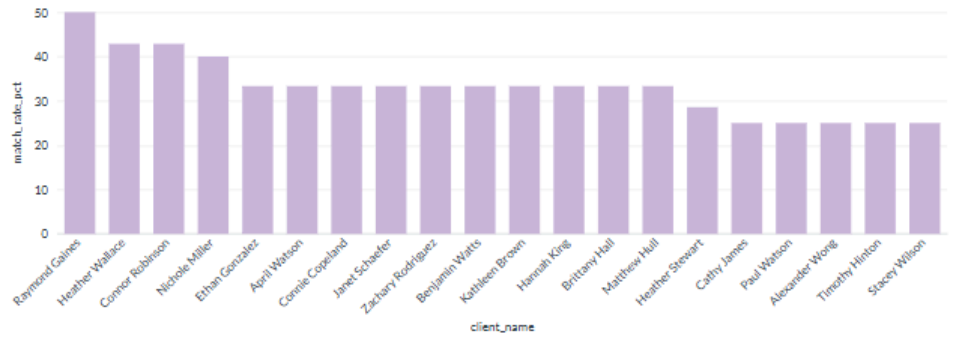


Client Preference Match Rate

client_id	client_name	total_inquiries	matched_inquiries	match_rate_pct
932	Raymond Gaines	4	2	50
909	Heather Wallace	7	3	42.86
796	Connor Robinson	7	3	42.86
440	Nichole Miller	5	2	40
732	Ethan Gonzalez	3	1	33.33
483	April Watson	3	1	33.33
134	Connie Copeland	3	1	33.33
743	Janet Schaefer	3	1	33.33
44	Zachary Rodriguez	3	1	33.33
927	Benjamin Watts	3	1	33.33
944	Kathleen Brown	3	1	33.33
382	Hannah King	3	1	33.33
277	Brittany Hall	3	1	33.33
886	Matthew Hull	6	2	33.33
629	Heather Stewart	7	2	28.57
668	Cathy James	4	1	25
186	Paul Watson	4	1	25
408	Alexander Wong	4	1	25
284	Timothy Hinton	4	1	25

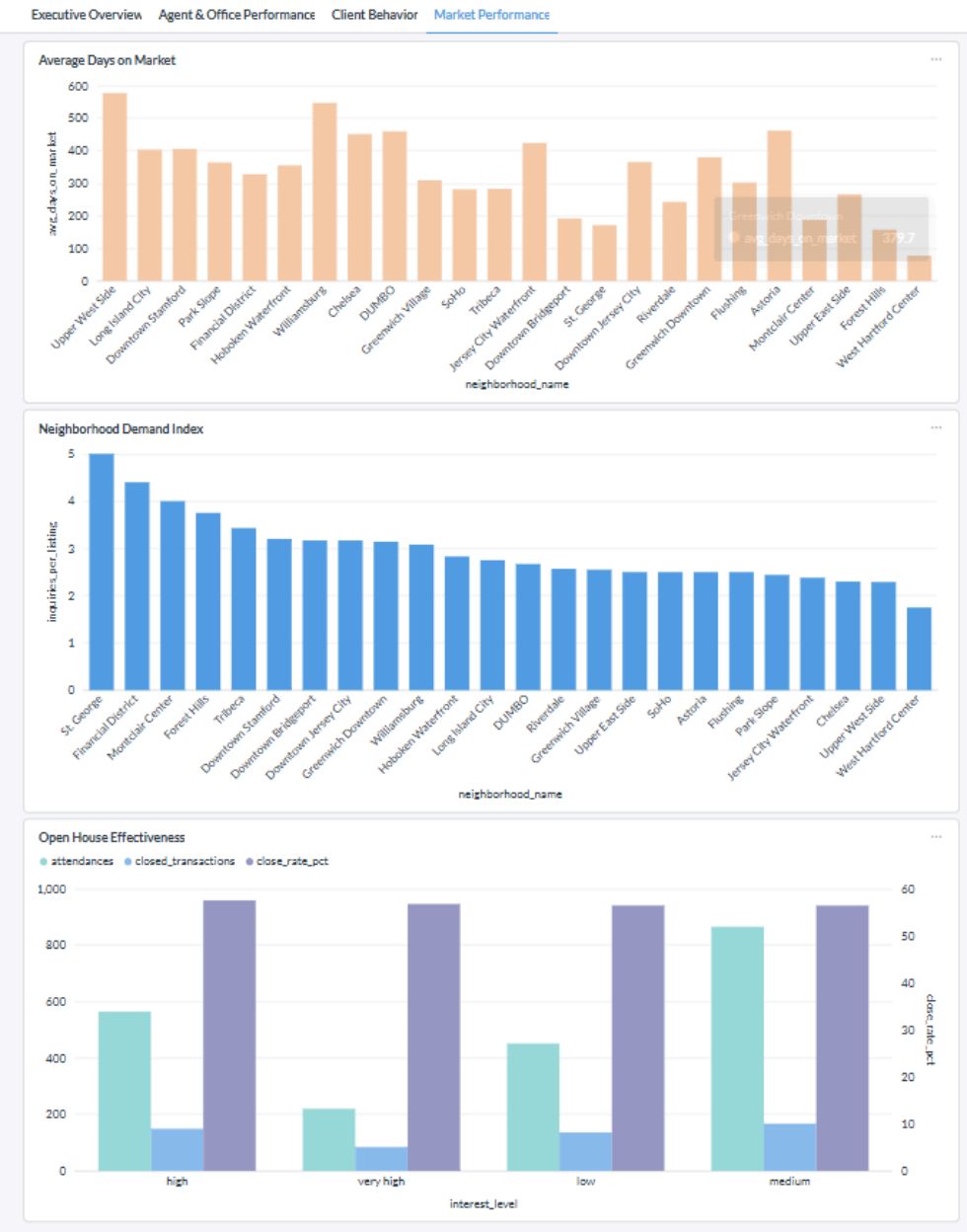
20 rows

Client Preference Match Rate Bar Chart



This dashboard analyzes client behavior across the sales pipeline, focusing on engagement levels, channel effectiveness, and preference alignment. It includes inquiry channel conversion, client engagement scoring, and client preference match rate. The results show that the website channel generates the highest volume of inquiries and the strongest conversion performance, while walk-in inquiries have relatively lower conversion rates. The dashboard also identifies highly engaged clients based on inquiries, appointments, and open house attendance, with top engagement scores around 16–20. In addition, it shows that top clients have preference match rates of about 40% to 50%, while most clients fall around 25% to 35%, suggesting room to improve recommendation accuracy and listing targeting. This dashboard is mainly designed for marketing teams and sales strategists, supporting decisions related to channel optimization, lead prioritization, and client targeting.

9.4 Dashboard 4: Market Performance



This dashboard evaluates market performance across neighborhoods and listing strategies, focusing on property liquidity, demand intensity, and the effectiveness of open house events. The “Average Days on Market” chart shows how long properties remain listed before being sold across neighborhoods, helping identify areas where pricing or marketing strategies may need adjustment. The “Neighborhood Demand Index” measures demand intensity by comparing inquiries per listing and highlights areas with stronger buyer interest relative to supply. The “Open House Effectiveness” chart shows that while higher attendance generally correlates with increased transaction volume, the conversion rate remains relatively stable across interest levels, suggesting that open houses are effective for engagement but may need stronger follow-up to

improve conversion. This dashboard is designed for operations managers, marketing teams, and real estate strategists, supporting decisions on pricing, inventory allocation, marketing effectiveness, and high-demand neighborhood targeting.

10. Conclusion

10.1 Summary of Goals Achieved

The goal of this project was to design a relational database system for Dream Homes NYC that could organize business data in a structured and queryable way. The final project delivered a normalized PostgreSQL database design, a loaded dataset aligned with the schema, analytical procedures for business insight generation, and a dashboard plan for executive reporting.

10.2 Insights Enabled

The database made it possible to examine several business questions that would be difficult to answer from disconnected files alone, including how effectively listings convert into closed transactions, how agent performance differs across listings and commissions, how client engagement changes across inquiries, appointments, and open houses, and how neighborhood activity varies across property and listing behavior.

10.3 Benefits of RDBMS, ETL & Structured Analysis

This project shows the value of a relational database approach compared with fragmented record keeping. Data integrity is improved through primary keys, foreign keys, and normalized relationships. Query power is improved because multiple business processes can be analyzed together through joins and aggregations. Scalability is improved because the system can support larger and more connected datasets than spreadsheet-based workflows. Automation is improved because ETL and loading steps can be repeated more consistently. Self-service analytics is improved because analysts can query the database directly while managers can use dashboards.

10.4 Future Recommendations

Several next steps could extend the value of this system, including more automated data ingestion, expanded dashboard functionality, and future forecasting or recommendation models. The database could also support additional operational modules if the client wants to expand into more detailed customer relationship or marketing workflows.

Appendices

GitHub Repository

All code (ETL, SQL queries, analysis notebooks):
https://github.com/RickAnalystCU-web/dream-homes-nyc